



SEPTEMBER 9–11

DevOps Foundations

From Code Commit to Production Confidence

- VAIBHAV GUJRAL



HOW THE NEXT FOUR HOURS WORK

One app. Six modules. Commit → Production



20 min

1. Culture & the Lifecycle

Why DevOps is a culture shift — the lifecycle, DORA metrics, and today's build map.



35 min

2. Source Control Done Right

Trunk-based flow, pull requests, branch protection. Lab: your first guarded merge.



45 min

3. CI: Build & Test Automation

GitHub Actions from scratch — build, test, artifact. Lab: watch CI block a bad change.



40 min

4. Infrastructure as Code

Terraform on Azure with OIDC. Lab: your whole environment from a pull request.



40 min

5. CD & Progressive Delivery

Environments, approvals, blue/green slot swaps, feature flags. Lab: ship to prod.



35 min

6. Production & SRE

Observability, SLOs, incident response. Demo: we break prod — and recover in 60s.

BEFORE WE START

Setup check – everyone green?



GitHub account + your fork

github.com/vaibhavgujral/kcdc-devops-foundations → Fork



Azure subscription

Free account or VS credits — total spend today: under \$1



Azure CLI signed in

`az login` · `az account show`



Git + VS Code

Optional but handy: .NET 10 SDK, Terraform CLI

EVERYTHING YOU NEED

Workshop repo + lab guide:



<https://bit.ly/4r5ekZh>

Builds run in GitHub Actions — your laptop only needs git, a browser, and az.



Culture & the Lifecycle

Why DevOps is a people problem wearing a technology costume.



MODULE 1 · A WORKING DEFINITION

DevOps, defined

“DevOps is the union of people, process, and products to enable continuous delivery of value to your end users.”

— Donovan Brown, Microsoft



People

Culture comes first: shared ownership, trust, and blameless learning. Tools can't fix incentives.



Process

Small batches, fast feedback, everything reviewed and automated — the loop we keep shrinking.



Products

The tooling that makes the process real: GitHub, Actions, Terraform, Azure — today's stack.

MODULE 1 · CULTURE

DevOps is not a thing you install

What it isn't

- ✗ A tool you buy or a pipeline you copy
- ✗ A team that sits between dev and ops
- ✗ A job title ("we hired a DevOps")
- ✗ Just CI/CD — that's necessary, not sufficient

What it is

- ✓ Shared ownership: build it, run it, own it
- ✓ Fast feedback loops at every stage
- ✓ Small batches, shipped often, easily undone
- ✓ A learning culture — failure is data, not blame

MODULE 1 · CULTURE

The wall of confusion



Development

Measured on change.

Ship features fast, respond to the market, keep stakeholders happy.

T H E W A L L



Operations

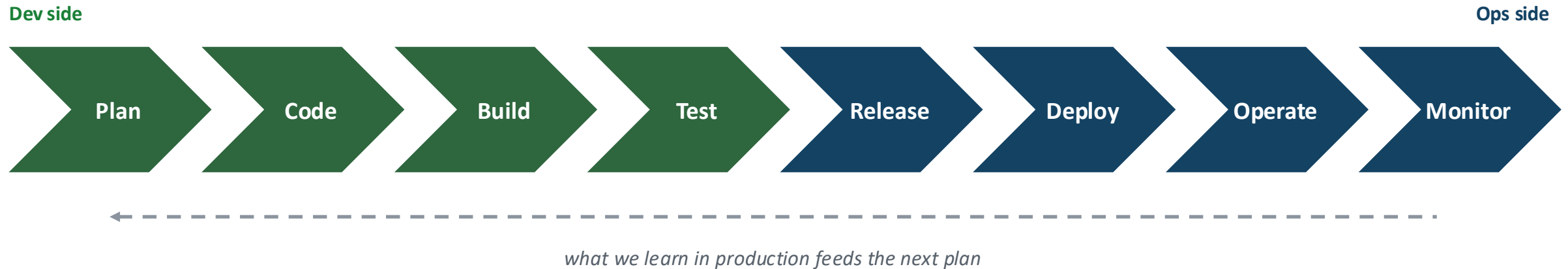
Measured on stability.

Keep it up, keep it safe — and every change is a threat to both.

DevOps tears down the wall: one team, shared goals, shared pager. “You build it, you run it.” — Werner Vogels, Amazon CTO

MODULE 1 · THE LIFECYCLE

Not a line – a loop



The loop is the product

Elite teams go around it many times a day, not once a quarter.



Shrink the loop

Every practice today — CI, IaC, flags, observability — exists to make one trip around faster and safer.



Close the loop

Monitoring isn't the end. It's the input to the next iteration.

MODULE 1 · THE PLAN STAGE

Where the loop starts: planning



The Agile mindset

Individuals and interactions over processes; responding to change over following a plan. DevOps is Agile carried all the way to production.



Scrum

Fixed-length sprints from a prioritized backlog, with a review and retrospective every cycle — cadence and inspection.



Kanban

Visualize the flow, limit work-in-progress, watch where cards pile up. Flow over ceremony.



GitHub Projects & Issues

Plan where the code lives: issues link to branches, PRs close issues, boards update themselves.

Today's four hours start at the commit — but elite delivery starts earlier, with small, well-sliced work items.

MODULE 1 · MEASURING IT

The four DORA keys (classic)



Deployment frequency

On-demand

How often you ship. Elite: multiple deploys per day.



Lead time for changes

< 1 day

Commit → running in production. Elite: under a day.



Change failure rate

< 15%

Share of deploys causing incidents or rollback.



Time to restore

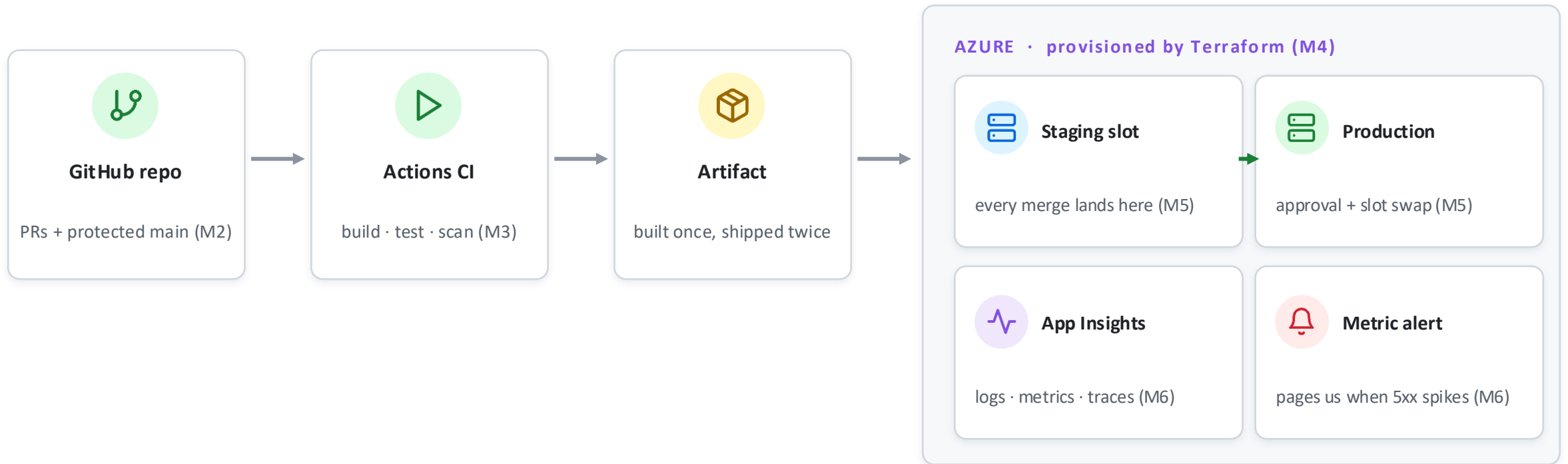
< 1 hour

When it breaks — and it will — how fast is recovery?

The research headline: speed and stability rise together, not a trade-off. These are the classic four keys; DORA added a fifth (deployment rework rate) in 2024, and thresholds shift yearly — treat these as direction, not law.

MODULE 1 · THE BUILD MAP

Today: one commit, end to end



The demo app: “QuoteBoard” — a small ASP.NET Core site. Small on purpose: the pipeline is the product today.



Source Control Done Right

Branching that scales, pull requests that get reviewed, and a main branch that protects itself



MODULE 2 · GIT ESSENTIALS

Git in four verbs – a 90 second refresher



commit

A snapshot of your changes plus a message saying why. The atom of history.



branch

An independent line of work — cheap to create, safe to experiment on, easy to throw away.



push

Send your local commits to the shared remote so the team (and CI) can see them.



pull

Fetch and merge everyone else's work into yours. Do it often; conflicts stay tiny.

Git is the distributed version-control system on your machine — every clone holds the full history. **GitHub** is the cloud home for those repos plus everything collaborative: pull requests, reviews, Actions, branch protection — the layer this whole workshop lives in.

MODULE 2 · BRANCHING

Pick a strategy: GitFlow vs. trunk-based

GitFlow

- Long-lived develop / release / hotfix branches
- Integration deferred → merge conflicts pile up
- Heavy ceremony per release
- **Fits: versioned, shrink-wrapped software**

Trunk-based

- Short-lived branches off main — integrated at least daily, often lasting only hours
- Integrate continuously → conflicts stay tiny
- Unfinished work hides behind feature flags
- **Fits: continuous delivery — today's model**

DORA finding: trunk-based development with small, short-lived branches is a statistical predictor of elite delivery performance.

MODULE 2 · CODE REVIEW

Pull requests people actually review



Keep them small

Under ~400 lines. Review quality collapses past that — big PRs get rubber-stamped, not reviewed.



Describe the why

The diff shows what changed. The description carries intent, trade-offs, and test notes.



Review as conversation

Ask questions, suggest, teach. Gatekeeping kills throughput and morale in equal measure.



Automate the nitpicks

Formatters and linters argue about whitespace so humans can review design and correctness.

Small PRs are a kindness to your reviewer — and the single cheapest quality win available to any team.

MODULE 2 · GUARDRAILS

Branch protection: main defends itself



Require a pull request

No direct pushes to main — every change gets a second look, even yours.



Require status checks

CI must be green before merge is possible. We wire this in Lab 2.



Block force pushes

History is append-only. Nobody rewrites main at 2 AM.



CODEOWNERS

Changes to /infra auto-request the platform team. Review routing as code.

WHAT THIS BUYS YOU

main is always releasable.

That one property unlocks everything after it: continuous deployment only works if the thing you continuously deploy is trustworthy.

Guardrails aren't distrust of people — they're respect for 2 AM, for new joiners, and for future you.

Your first guarded merge

- 1 Protect main** Settings → Rules → new branch ruleset: require PRs, block force pushes
- 2 Try to break it** Push straight to main — watch the server say no
- 3 Branch + change** feature/add-your-name → add yourself to CONTRIBUTORS.md
- 4 Open a real PR** Description says why, not just what
- 5 Merge + clean up** Merge, delete the branch — short-lived, as promised
- 6 Peek at CODEOWNERS** See how /infra changes would auto-route to owners



Fell behind?

Fast-forward from the checkpoint branch:

`checkpoint/lab-1-complete`

Full commands + code are in the lab guide — build from there, not from the slide.



CI: Build & Test Automation

Every commit builds. Every build tests. One artifact ships everywhere. From zero to a working pipeline.



MODULE 3 · CONTINUOUS INTEGRATION

Four Rules of CI



Integrate on every commit

Every push, every PR — no exceptions, no “skip CI just this once.”



Build once, deploy many

One artifact travels through every environment. Rebuild for prod = deploy something you never tested.



Fail fast, fix fast

Keep the loop under ~10 minutes. A red build is the team's top priority — nothing merges onto broken.



The build is the truth

“Works on my machine” dies here. If CI can't build it, it isn't done.

MODULE 3 · GITHUB ACTIONS

Anatomy of a workflow

```
name: CI
on:
  # ◀ trigger
  push: { branches: [main] }
  pull_request: { branches: [main] }

jobs:
  build-and-test:
    # ◀ job
    runs-on: ubuntu-latest # ◀ runner
    steps:
      # ◀ steps
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
        with: { dotnet-version: '10.0.x' }
      - run: dotnet build -c Release
      - run: dotnet test -c Release
      - run: dotnet publish -o publish
      - uses: actions/upload-artifact@v4
        with: { name: webapp, path: publish }
```



Trigger (on:)

Which events start the run — pushes, PRs, schedules, manual dispatch.



Job + runner

A fresh, disposable VM per job. Nothing survives between runs — reproducibility by design.



Steps

Shell commands plus reusable actions from the marketplace, sharing one workspace.



Artifact

The build output we keep — and deploy in Lab 4. YAML lives in the repo, versioned like code.

MODULE 3 · BEYOND THE BASICS

Four features you'll use constantly



Matrix builds

One job definition fans out across versions or platforms in parallel — .NET 10 and 9, Linux and Windows, in one block.



Caching

Reuse restored packages between runs. Minutes off every build — remember the 10-minute rule.



Artifacts

Files that outlive the disposable runner: build outputs, test reports, coverage. The bridge from CI to CD.



Reusable workflows

Write the pipeline once, call it from every repo. How platform teams scale standards without policing.

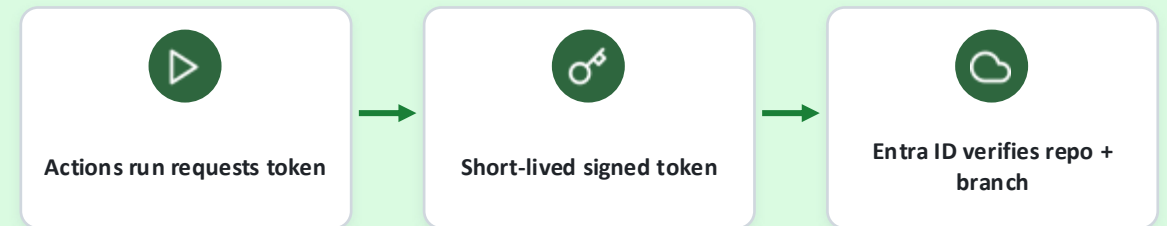
MODULE 3 · IDENTITY

Secrets, and the best secret of all

The old way: stored credentials

- Service-principal secret pasted into GitHub
- Long-lived → leakable, rotatable, forgettable
- Expires on a Saturday, breaks every deploy
- **Leaked CI credentials are a top breach vector**

Today's way: OIDC federation



Azure trusts “repo X, branch main” — not a password. The token lives minutes and is minted per run.

No long-lived Azure secret to store or rotate.

“The best secret is the one that doesn't exist.” — We set this up in Lab 3, and never store an Azure credential all day.

MODULE 3 · TEST STRATEGY

Know your tests



Unit

The smallest testable pieces, in isolation. Milliseconds each — you want hundreds. What CI runs in Lab 2.



Integration

Components together with real dependencies — the seams where systems actually break.



Smoke

A fast “is it alive?” after every deploy. Our curl /health in Lab 4 is exactly this.



Acceptance

Does it meet the requirement, at user level? Fewest in number, slowest, closest to the customer.

Shape it like a pyramid: many fast unit tests at the base, fewer slower tests as you climb. Invert it and your pipeline gets slow, flaky, and ignored — the 10-minute rule dies first.

MODULE 3 · QUALITY GATES

Shift left: quality is a pipeline stage



Tests on every PR

The gate you'll feel in Lab 2 — a failing test physically blocks the merge button.



Static analysis (CodeQL)

When enabled and configured, scans PRs for many injection and path-traversal patterns before a human looks — a strong net, not a guarantee.



Dependabot

Automated PRs when dependencies age or turn vulnerable. Your supply chain, on autopilot.



Secret scanning + push protection

GitHub blocks the push if a credential is in the diff. The cheapest incident you'll never have.

Finding it in the PR costs minutes. Finding it in production costs an incident. That's the entire economics of shift-left.

Build the CI pipeline – then break it

- 1 Write ci.yml** Trigger → checkout → setup .NET → build → test → publish → artifact
- 2 Ship it via PR** Watch the Checks tab run your pipeline live
- 3 Make it required** Add build-and-test to the ruleset — CI now guards the merge button
- 4 Sabotage a test** Break QuoteProvider, push a PR — savor the red  and the blocked merge
- 5 Stretch goals** NuGet caching · a .NET 10/9 matrix · enable CodeQL + Dependabot



Fell behind?

Fast-forward from the checkpoint branch:

[checkpoint/lab-2-complete](#)

Full commands + code are in the lab guide — build from there, not from the slide.



Infrastructure as Code

Your entire environment, defined in code, reviewed in pull requests, applied by the pipeline.



MODULE 4 · WHY IAC?

ClickOps doesn't scale. Code does.

Portal clicking ("ClickOps")

- Snowflake environments no one can reproduce
- No history, no review, no rollback
- "Who changed the firewall?" — silence
- Drift accumulates invisibly until it bites
- Knowledge lives in one person's head

Infrastructure as Code

- Versioned in git, reviewed in PRs — like app code
- Environments are repeatable: dev = staging = prod
- Disposable: rebuild from scratch in minutes
- Self-documenting — the code is the diagram
- **Pets → cattle: replace, don't nurse**

MODULE 4 · TERRAFORM

The mental model: declare, diff, converge



You declare the end state

HCL describes what should exist — not the steps to create it. Terraform figures out the how.



plan shows the diff

“7 to add, 0 to change, 0 to destroy.” A reviewable preview of every change — before it happens.



apply converges reality

Terraform calls Azure's APIs until reality matches the code. Run it twice: no changes — idempotent.



State is its memory

A state file maps your code to real resource IDs. Ours lives in Azure Storage, shared and locked.

```
resource "azurerm_linux_web_app" "web" {
  name                       = "app-quoteboard-vg42"
  resource_group_name       = azurerm_resource_group.app.name
  location                  = var.location
  service_plan_id           = azurerm_service_plan
                             .plan.id

  site_config {
    application_stack {
      dotnet_version = "10.0"
    }
    health_check_path = "/health"
  }
}

$ terraform plan
Plan: 7 to add, 0 to change,
    0 to destroy.
```

MODULE 4 · THE WORKFLOW

Infrastructure changes ride the same rails as code



Review the blast radius

The plan output is the review artifact — you approve exactly what will change, nothing more.



Drift detection for free

Someone clicked in the portal? The next plan surfaces the difference between code and reality.



Destroy is a feature

terraform destroy tears the whole environment down cleanly — which is exactly how today ends.

Provision in Azure from a pull request

- 1 **Bootstrap (done at break)** OIDC app + federated credentials + state storage — Lab 3.1 script
- 2 **Write the Terraform** RG · S1 plan · web app + staging slot · App Insights · 5xx alert
- 3 **Add infra.yml** plan on PRs, apply on merge to main — OIDC login, no secrets
- 4 **PR it — read the plan** “Plan: 7 to add” — that diff is your review artifact
- 5 **Merge and verify** Watch apply run, then browse your (empty) site on azurewebsites.net



Fell behind?

Fast-forward from the checkpoint branch:

[checkpoint/lab-3-complete](#)

Full commands + code are in the lab guide — build from there, not from the slide.



CD & Progressive Delivery

Every merge reaches staging. A human approves. Production swaps in with zero downtime — and rollback is a button.



MODULE 5 · DEFINITIONS

Continuous delivery vs continuous deployment



Continuous DELIVERY (today)

Every merge is releasable and staged automatically; a human chooses when production happens. The click replaces the change-advisory-board meeting.

Continuous DEPLOYMENT

Remove the gate: green pipeline → production, no human at all. Same machinery — the gate is one line of YAML. Earn it with test confidence and observability.

MODULE 5 · GITHUB ENVIRONMENTS

Environments: gates, scopes, and history



Required reviewers

The approval gate. A pipeline pauses until a designated human approves — audit-logged, one click.



Scoped secrets & variables

Production credentials exist only inside the production environment. Staging jobs can't touch them.



Wait timers & branch rules

Bake time before prod, and only main may deploy — policy expressed as configuration.



Deployment history + URL

Who shipped what, when, from which commit — with a live link on every run. Your audit trail, for free.

In Lab 4 you become your own change advisory board — same governance, none of the meetings.

MODULE 5 · PROGRESSIVE DELIVERY

Three ways to ship without fear



Rolling

Replace instances a few at a time until the fleet is new.

- Simple, resource-cheap
- Rollback = roll forward again (slow)
- Two versions live during rollout



Blue/green — today's lab

Two identical environments; warm the new one, then flip the router.

- Zero-downtime cutover
- Instant rollback: flip back
- Azure slots make this one command



Canary

Send a small % of real traffic to the new version; watch, then expand.

- Real users validate safely
- Blast radius = the canary %
- Needs metrics to judge — module 6

MODULE 5 · PROGRESSIVE EXPOSURE

Widening the circle: rings, dark launches, A/B



Deployment rings

Expand the blast radius in deliberate waves: canary users → early adopters → everyone.

- How Microsoft ships Windows and Azure DevOps
- Each ring bakes before the next opens
- Rings are audiences; slots and flags are the mechanics



Dark launching

The feature is deployed and running in production — but invisible to users.

- Validate performance under real load, zero exposure
- The reveal is a flag flip, not a deploy
- How we shipped the QuoteBoard banner in Lab 4



A/B testing

Two variants, real users, and the data decides which one wins.

- Releases become experiments
- Needs metrics + enough traffic to judge
- Flags choose the variant; telemetry scores it

MODULE 5 · FEATURE FLAGS

Deploy ≠ release



Ship dark

Merge unfinished work safely behind a flag — the enabler of trunk-based development from module 2.



Release by config

Turning a feature on is a setting change, not a deployment. Marketing picks the moment, not the pipeline.



Instant kill switch

Feature misbehaving? Off in seconds — no rollback, no redeploy, no war room.



Progressive exposure

Internal users → 5% → everyone. Canary releasing at the feature level, not the deploy level.

```
// QuoteBoard homepage
var banner =
  cfg.GetValue<bool>(
    "FEATURE_NEW_BANNER")
    ? RenderBanner()
    : "";

# release, without deploying:
$ az webapp config appsettings \
  set --settings \
    FEATURE_NEW_BANNER=true
```

Flag hygiene: flags are temporary. Retire them or inherit a config minefield.

Ship it: staging → approval → production

- 1 **Create environments** staging (open) and production (required reviewer: you)
- 2 **Write deploy.yml** build → deploy staging slot → smoke test → gate → slot swap
- 3 **Merge and watch** Staging deploys itself; the run pauses at “Review pending deployments”
- 4 **Approve production** One click → warm swap → zero-downtime release. You're live 🎉
- 5 **Flip the flag** az CLI: FEATURE_NEW_BANNER=true — a release with no deployment
- 6 **Stretch: canary** Route 10% of prod traffic to staging with traffic-routing



Fell behind?

Fast-forward from the checkpoint branch:

[checkpoint/lab-4-complete](#)

Full commands + code are in the lab guide — build from there, not from the slide.



Production & SRE

You shipped it. Now you run it: observability, SLOs, incidents — and breaking production on purpose.



MODULE 6 · OBSERVABILITY

The three pillars — and where they live in Azure



Logs

What happened

Discrete, timestamped events with full context. Rich but voluminous — where you go to understand a specific failure.



Metrics

How much, how fast

Cheap numeric aggregates over time: request rate, latency, 5xx count. What you alert on and trend.



Traces

Where time went

One request's journey across services. Finds the slow hop in a distributed system.

Application Insights gives us all three — auto-instrumented for .NET, plus Live Metrics (a real-time firehose) and the Failures blade. Terraform already wired it up in Lab 3; telemetry has been flowing since your first deploy.

MODULE 6 · RELIABILITY TARGETS

SLIs, SLOs, and the error budget



SLI — the measurement

A metric users actually feel: share of successful requests, p95 latency. Not CPU. Users don't feel CPU.



SLO — the target

“99.9% of requests succeed, measured monthly.” A promise you design and staff around. (An SLA is its contractual cousin: an SLO with penalties attached.)



Error budget — the license

100% minus the SLO. Budget left? Ship boldly. Budget spent? Slow down and invest in reliability.

A 99.9% MONTHLY SLO BUYS YOU

43

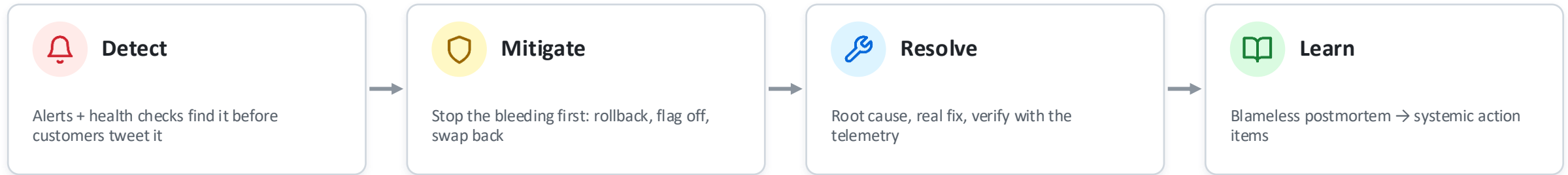
minutes of downtime — per month

Chasing 100% is chasing infinity: each extra nine gets sharply more expensive (often cited loosely as ~10×) and tips into change-freeze paralysis.

The error budget ends the dev-vs-ops war with arithmetic: it legitimizes risk-taking — up to a number.

MODULE 6 · WHEN IT BREAKS

Incidents and blameless learning



Mitigate first, diagnose later

Users don't care about root cause while they're down. Restore service, then investigate.



Humans are never the root cause

“Engineer made a typo” isn't a finding. “Nothing caught the typo” is. Fix systems, not people.



Action items that change the system

“Be more careful” is not an action item. “Add a canary stage” is.

MODULE 6 · BEYOND TODAY

Shift right: validating where it counts



Testing in production

Some conditions only exist in prod: real traffic, real data shapes, real failure modes. Staged rollouts, synthetic monitors, and chaos experiments test there — safely.



Measure the response: TTD · TTM · TTR

Our terms — time to detect, mitigate, remediate — break down recovery into clocks you can watch. DORA now frames this as failed deployment recovery time (it retired the ambiguous MTTR); our demo times detect + mitigate live.



Security observability

War-game exercises, live-site penetration tests, continuous monitoring for abnormal patterns. Shift-left's production-side twin.



Platform engineering

Golden paths and self-service: a platform team packages today's entire pipeline so every product team gets it without rebuilding it. Where mature DevOps orgs are heading.

Shift left finds bugs cheaply. Shift right finds truths that only production can tell you. Elite teams do both.

LAB 5

~10 min

Let's break production. On purpose.



1. Inject failure `FAIL_RATE=0.4` — the app now throws 500s on 40% of requests



2. Generate traffic Everyone, hammer the demo URL — be the chaos



3. Watch it burn, calmly Live Metrics spikes → Failures blade → the Terraform alert fires



4. Mitigate in ~60 seconds Config rollback (`FAIL_RATE=0`) — or slot-swap right back



5. Sketch the postmortem Timeline, impact, contributing causes, systemic action items

THE POINT

MTTR is a design decision, not a heroic effort.

Confidence isn't the absence of failure — it's knowing exactly what you'll do when it shows up.

WRAP UP

What you built in four hours



A repo that defends itself Protected main, PR flow, CODEOWNERS



CI on every change Build, test, artifact — and a blocked bad merge to prove it



An environment born from a PR Terraform + OIDC: reviewable, repeatable, disposable



A gated path to production Staging → approval → zero-downtime swap → flag flip



A system you can operate Telemetry, alerts, and a rehearsed 60-second recovery

Don't forget: terraform destroy before you leave — Terraform giveth, Terraform taketh away.

WRAP UP

Where to go from here?

Read & measure



Accelerate — Forsgren, Humble, Kim
the research behind everything we did today



The Phoenix Project — Kim et al.
the culture shift, as a novel you'll finish in a weekend



Google SRE book (free online)
SLOs, error budgets, toil — module 6, in depth



DORA State of DevOps report
benchmark your team against the industry, yearly

Build next



Containers + AKS
same pipeline patterns, orchestrated fleets



GitOps (Flux / Argo CD)
the cluster pulls desired state from git — IaC's next step



Policy as code
OPA / Azure Policy: compliance as a status check



Chaos engineering
today's FAIL_RATE demo, as a discipline (Azure Chaos Studio)

About me



Based out of Omaha, NE since 2016

Principal Architect at Capgemini

7x Microsoft Most Valuable Professional (MVP)

Co-organizer, Omaha Azure User Group



Omaha Azure
User Group



LinkedIn



Twitter



Blog



Microsoft
**Most Valuable
Professional**



Titanium Sponsors



Platinum

Sponsors

Gold

Sponsors



Friends of KCDC



Other Awesome Organizations





SESSION FEEDBACK